Week 10 Report

Computational Statistics for Data Science · ISS507700

Student ID: 112006234

May 3, 2026

I have largely solved the problems without the use of any AI agent or tool.

Q0 · Setup

```
# load data
cars <- read.table(
  "auto-data.txt",
  header = FALSE,
  na.strings = "?",
  quote = "\""
)

colnames(cars) <- c(
  "mpg", "cylinders", "displacement", "horsepower",
  "weight", "acceleration", "model_year", "origin", "car_name"
)

# remove rows with missing values
cars <- na.omit(cars)

# remove car name because it is not used in regression
cars$car_name <- NULL
```

Q1 · Create a new dataset that log-transforms several variables from our original dataset (called cars in this case):

```r
# create log-transformed dataset
cars_log <- with(cars, data.frame(
  log.mpg. = log(mpg),
  log.cylinders. = log(cylinders),
  log.displacement. = log(displacement),
  log.horsepower. = log(horsepower),
  log.weight. = log(weight),
  log.acceleration. = log(acceleration),
  model_year = model_year,
  origin = origin
))
```

(a) Run a new regression on the cars_log dataset, with mpg.log. dependent on all other variables.

```r
regr_log <- lm(
  log.mpg. ~ log.cylinders. + log.displacement. + log.horsepower. +
    log.weight. + log.acceleration. + model_year + factor(origin),
  data = cars_log
)

summary(regr_log)
```

```
#>
#> Call:
#> lm(formula = log.mpg. ~ log.cylinders. + log.displacement. +
#>     log.horsepower. + log.weight. + log.acceleration. + model_year +
#>     factor(origin), data = cars_log)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.39727 -0.06880  0.00450  0.06356  0.38542
#>
#> Coefficients:
#>                    Estimate Std. Error t value Pr(>|t|)
#> (Intercept)        7.301938   0.361777  20.184  < 2e-16 ***
#> log.cylinders.    -0.081915   0.061116  -1.340  0.18094
#> log.displacement.  0.020387   0.058369   0.349  0.72707
#> log.horsepower.   -0.284751   0.057945  -4.914 1.32e-06 ***
#> log.weight.       -0.592955   0.085165  -6.962 1.46e-11 ***
#> log.acceleration. -0.169673   0.059649  -2.845  0.00469 **
#> model_year         0.030239   0.001771  17.078  < 2e-16 ***
#> factor(origin)2    0.050717   0.020920   2.424  0.01580 *
#> factor(origin)3    0.047215   0.020622   2.290  0.02259 *
```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.113 on 383 degrees of freedom
#> Multiple R-squared:  0.8919, Adjusted R-squared:  0.8897
#> F-statistic:   395 on 8 and 383 DF,  p-value: < 2.2e-16
```

(i) Which log-transformed factors have a significant effect on log.mpg. at 10% significance?

```
coef_table <- summary(regr_log)$coefficients

# significant variables at 10% level
coef_table[coef_table[, "Pr(>|t|)"] < 0.10, ]
```

```
#>                     Estimate  Std. Error   t value     Pr(>|t|)
#> (Intercept)        7.30193842 0.361776739 20.183549 3.164927e-62
#> log.horsepower.   -0.28475129 0.057944973 -4.914167 1.324103e-06
#> log.weight.       -0.59295537 0.085164828 -6.962444 1.461217e-11
#> log.acceleration. -0.16967287 0.059649081 -2.844518 4.686605e-03
#> model_year         0.03023926 0.001770634 17.078210 5.044732e-49
#> factor(origin)2    0.05071723 0.020919628  2.424385 1.579639e-02
#> factor(origin)3    0.04721481 0.020621538  2.289587 2.258853e-02
```

At the 10 percent significance level, log horsepower, log weight, log acceleration , model year, and factor(origin) are statistically significant predictors of log mpg. The variables log cylinders. and log displacement are not significant at the 10 percent level.

At the 10% significance level, the significant factors are the predictors with p-value < 0.10 in the log-transformed regression output. These variables have statistically significant relationships with log(mpg), after controlling for the other predictors in the model.

(ii) Do some new factors now have effects on mpg, and why might this be?

```
# original full regression model, without log transformation
regr_raw <- lm(
  mpg ~ cylinders + displacement + horsepower +
    weight + acceleration + model_year + factor(origin),
  data = cars
)

summary(regr_raw)
```

```
#>
#> Call:
#> lm(formula = mpg ~ cylinders + displacement + horsepower + weight +
#>     acceleration + model_year + factor(origin), data = cars)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -9.0095 -2.0785 -0.0982  1.9856 13.3608
#>
#> Coefficients:
#>                   Estimate Std. Error t value Pr(>|t|)
#> (Intercept)     -1.795e+01  4.677e+00  -3.839 0.000145 ***
#> cylinders       -4.897e-01  3.212e-01  -1.524 0.128215
#> displacement     2.398e-02  7.653e-03   3.133 0.001863 **
#> horsepower      -1.818e-02  1.371e-02  -1.326 0.185488
#> weight          -6.710e-03  6.551e-04 -10.243  < 2e-16 ***
#> acceleration     7.910e-02  9.822e-02   0.805 0.421101
#> model_year       7.770e-01  5.178e-02  15.005  < 2e-16 ***
#> factor(origin)2  2.630e+00  5.664e-01   4.643 4.72e-06 ***
#> factor(origin)3  2.853e+00  5.527e-01   5.162 3.93e-07 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 3.307 on 383 degrees of freedom
#> Multiple R-squared:  0.8242, Adjusted R-squared:  0.8205
#> F-statistic: 224.5 on 8 and 383 DF,  p-value: < 2.2e-16
```

```
summary(regr_log)
```

```
#>
#> Call:
#> lm(formula = log.mpg. ~ log.cylinders. + log.displacement. +
#>     log.horsepower. + log.weight. + log.acceleration. + model_year +
#>     factor(origin), data = cars_log)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.39727 -0.06880  0.00450  0.06356  0.38542
#>
#> Coefficients:
#>                    Estimate Std. Error t value Pr(>|t|)
#> (Intercept)        7.301938   0.361777  20.184  < 2e-16 ***
#> log.cylinders.    -0.081915   0.061116  -1.340  0.18094
#> log.displacement.  0.020387   0.058369   0.349  0.72707
#> log.horsepower.   -0.284751   0.057945  -4.914 1.32e-06 ***
#> log.weight.       -0.592955   0.085165  -6.962 1.46e-11 ***
#> log.acceleration. -0.169673   0.059649  -2.845  0.00469 **
```

```
#> model_year         0.030239   0.001771  17.078  < 2e-16 ***
#> factor(origin)2    0.050717   0.020920   2.424  0.01580 *
#> factor(origin)3    0.047215   0.020622   2.290  0.02259 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.113 on 383 degrees of freedom
#> Multiple R-squared:  0.8919, Adjusted R-squared:  0.8897
#> F-statistic:   395 on 8 and 383 DF,  p-value: < 2.2e-16
```

Yes. After the log transformation, some new factors now have significant effects on mpg. In the original raw regression, the significant predictors were mainly displacement, weight, model year, and origin. In the log-transformed regression, log horsepower. and log acceleration. also become significant at the 10 percent level. This suggests that horsepower and acceleration affect mpg in a nonlinear or proportional way, so their effects become clearer after taking logs.

This may happen because the raw regression assumes a constant additive effect, while the log regression estimates proportional effects. Since variables such as horsepower and acceleration likely have nonlinear relationships with mpg, the log transformation can reduce curvature and make their effects easier to detect statistically.

(iii) Which factors still have insignificant or opposite (from correlation) effects on mpg? Why might this be?

Look at the p-values in the log regression summary.

```
# regression output
summary(regr_log)
```

```
#>
#> Call:
#> lm(formula = log.mpg. ~ log.cylinders. + log.displacement. +
#>     log.horsepower. + log.weight. + log.acceleration. + model_year +
#>     factor(origin), data = cars_log)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.39727 -0.06880  0.00450  0.06356  0.38542
#>
#> Coefficients:
#>                    Estimate Std. Error t value Pr(>|t|)
#> (Intercept)        7.301938   0.361777  20.184  < 2e-16 ***
#> log.cylinders.    -0.081915   0.061116  -1.340  0.18094
#> log.displacement.  0.020387   0.058369   0.349  0.72707
#> log.horsepower.   -0.284751   0.057945  -4.914 1.32e-06 ***
```

```
#> log.weight.       -0.592955   0.085165  -6.962 1.46e-11 ***
#> log.acceleration. -0.169673   0.059649  -2.845  0.00469 **
#> model_year         0.030239   0.001771  17.078  < 2e-16 ***
#> factor(origin)2    0.050717   0.020920   2.424  0.01580 *
#> factor(origin)3    0.047215   0.020622   2.290  0.02259 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.113 on 383 degrees of freedom
#> Multiple R-squared:  0.8919, Adjusted R-squared:  0.8897
#> F-statistic:   395 on 8 and 383 DF,  p-value: < 2.2e-16
```

log cylinders. and log displacement. are still insignificant in the log regression model.

Compare the signs of the simple correlations with the signs of the regression coefficients above.

```
# correlations with mpg in the original data
round(cor(cars[, c(
  "mpg", "cylinders", "displacement", "horsepower",
  "weight", "acceleration", "model_year"
)], use = "pairwise.complete.obs"), 2)
```

```
#>                mpg cylinders displacement horsepower weight acceleration
#> mpg           1.00     -0.78        -0.81      -0.78  -0.83         0.42
#> cylinders    -0.78      1.00         0.95       0.84   0.90        -0.50
#> displacement -0.81      0.95         1.00       0.90   0.93        -0.54
#> horsepower   -0.78      0.84         0.90       1.00   0.86        -0.69
#> weight       -0.83      0.90         0.93       0.86   1.00        -0.42
#> acceleration  0.42     -0.50        -0.54      -0.69  -0.42         1.00
#> model_year    0.58     -0.35        -0.37      -0.42  -0.31         0.29
#>               model_year
#> mpg                 0.58
#> cylinders          -0.35
#> displacement       -0.37
#> horsepower         -0.42
#> weight             -0.31
#> acceleration        0.29
#> model_year          1.00
```

```
# correlations with log.mpg. in the log data
round(cor(cars_log[, c(
  "log.mpg.", "log.cylinders.", "log.displacement.",
  "log.horsepower.", "log.weight.",
  "log.acceleration.", "model_year"
)], use = "pairwise.complete.obs"), 2)
```

```
#>                  log.mpg. log.cylinders. log.displacement. log.horsepower.
#> log.mpg.            1.00          -0.82             -0.86           -0.85
#> log.cylinders.     -0.82           1.00              0.95            0.83
#> log.displacement.  -0.86           0.95              1.00            0.87
#> log.horsepower.    -0.85           0.83              0.87            1.00
#> log.weight.        -0.87           0.88              0.94            0.87
#> log.acceleration.   0.47          -0.50             -0.52           -0.72
#> model_year          0.58          -0.34             -0.33           -0.40
#>                  log.weight. log.acceleration. model_year
#> log.mpg.               -0.87              0.47       0.58
#> log.cylinders.          0.88             -0.50      -0.34
#> log.displacement.       0.94             -0.52      -0.33
#> log.horsepower.         0.87             -0.72      -0.40
#> log.weight.             1.00             -0.42      -0.29
#> log.acceleration.      -0.42              1.00       0.31
#> model_year             -0.29              0.31       1.00
```

log.displacement. has a negative simple correlation with log.mpg., but its coefficient in the full regression is positive, so it has an opposite effect from the correlation.

This likely happens because of multicollinearity. Cylinders, displacement, horsepower, and weight are strongly correlated with each other, so the model has difficulty separating their individual effects. A regression coefficient shows the effect of one variable while holding the others constant, not the simple two-variable relationship.

(b) Let's take a closer look at weight, because it seems to be a major explanation of mpg

(i) Create a regression (call it regr_wt) of mpg over weight from the original cars dataset

```r
regr_wt <- lm(mpg ~ weight, data = cars)

summary(regr_wt)
```

```
#>
#> Call:
#> lm(formula = mpg ~ weight, data = cars)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -11.9736  -2.7556  -0.3358   2.1379  16.5194
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 46.216524   0.798673   57.87   <2e-16 ***
```

```
#> weight      -0.007647   0.000258  -29.64   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 4.333 on 390 degrees of freedom
#> Multiple R-squared:  0.6926, Adjusted R-squared:  0.6918
#> F-statistic: 878.8 on 1 and 390 DF,  p-value: < 2.2e-16
```

This model estimates the raw linear relationship between car weight and mpg. The slope should be negative, meaning heavier cars tend to have lower mpg. However, this raw model may not fully satisfy regression assumptions if the relationship is nonlinear.

(ii) Create a regression (call it regr__wt__log) of log.mpg. on log.weight. from cars__log

```r
regr_wt_log <- lm(log.mpg. ~ log.weight., data = cars_log)

summary(regr_wt_log)
```

```
#>
#> Call:
#> lm(formula = log.mpg. ~ log.weight., data = cars_log)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.52321 -0.10446 -0.00772  0.10124  0.59445
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  11.5152     0.2365   48.69   <2e-16 ***
#> log.weight.  -1.0575     0.0297  -35.61   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.1651 on 390 degrees of freedom
#> Multiple R-squared:  0.7648, Adjusted R-squared:  0.7642
#> F-statistic:  1268 on 1 and 390 DF,  p-value: < 2.2e-16
```

This model estimates the relationship between log(weight) and log(mpg). Since both variables are log-transformed, the model measures a proportional relationship instead of a fixed-unit relationship.

(iii) Visualize the residuals of both regression models (raw and log-transformed):

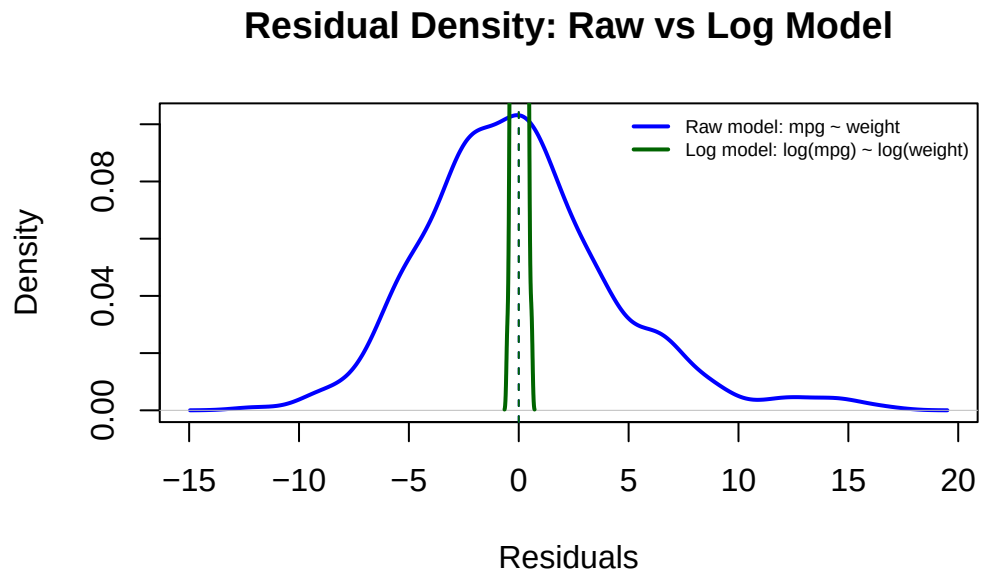
1. density plots of residuals

```r
plot(
  density(regr_wt$residuals),
  main = "Residual Density: Raw vs Log Model",
  xlab = "Residuals",
  lwd = 2,
  col = "blue"
)

lines(
  density(regr_wt_log$residuals),
  lwd = 2,
  col = "darkgreen"
)

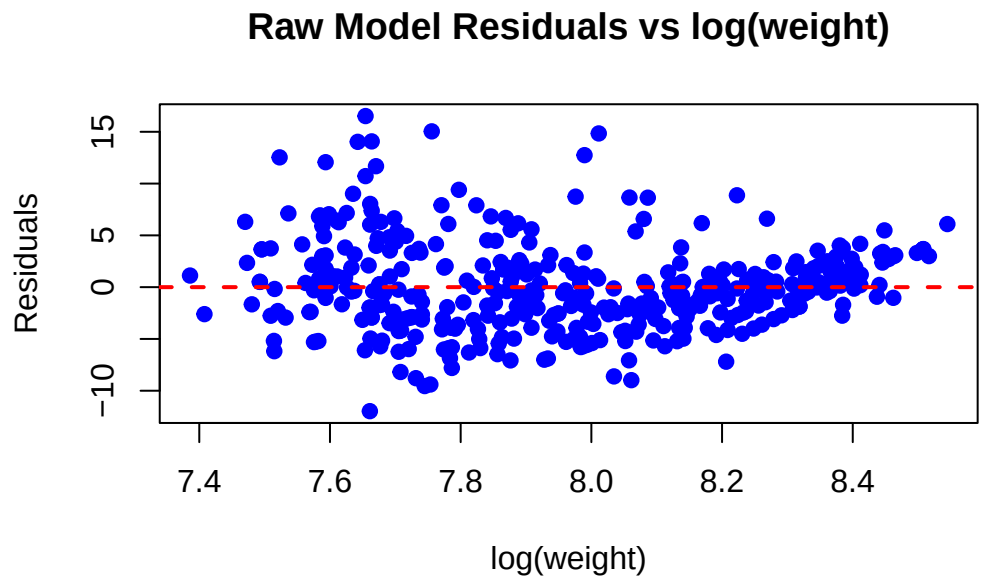
abline(v = mean(regr_wt$residuals), col = "blue", lty = 2)
abline(v = mean(regr_wt_log$residuals), col = "darkgreen", lty = 2)

legend(
  "topright",
  legend = c("Raw model: mpg ~ weight", "Log model: log(mpg) ~ log(weight)"),
  col = c("blue", "darkgreen"),
  lwd = 2,
  lty = 1,
  bty = "n",
  cex = 0.6
)
```

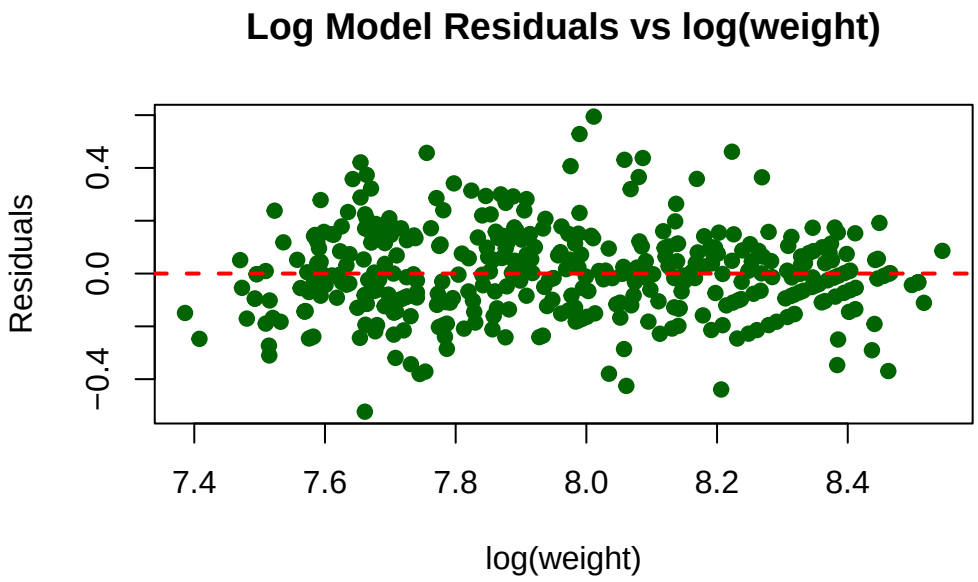


2. scatterplot of log.weight. vs. residuals

```
# raw model residuals
plot(
  cars_log$log.weight.,
  regr_wt$residuals,
  main = "Raw Model Residuals vs log(weight)",
  xlab = "log(weight)",
  ylab = "Residuals",
  pch = 19,
  col = "blue"
)
abline(h = 0, col = "red", lwd = 2, lty = 2)
```




```
# log-log model residuals
plot(
  cars_log$log.weight.,
  regr_wt_log$residuals,
  main = "Log Model Residuals vs log(weight)",
  xlab = "log(weight)",
  ylab = "Residuals",
  pch = 19,
  col = "darkgreen"
)
abline(h = 0, col = "red", lwd = 2, lty = 2)
```



The density plots show whether the residuals are roughly symmetric and centered around zero. The residual scatterplots show whether there is a systematic pattern. A good regression model should have residuals randomly scattered around zero, without a strong curve or funnel shape.

(iv) Which regression produces better distributed residuals for the assumptions of regression?

> The log-log regression produces better distributed residuals for the assumptions of regression.

The log-log regression produces better residuals because the residual scatterplot looks more randomly spread around zero and has less obvious curvature than the raw model. Its residual density also looks more symmetric and centered.

(v) How would you interpret the slope of log.weight. vs log.mpg. in simple words?

> Heavier cars tend to be less fuel efficient.

```
coef_table_log_wt <- summary(regr_wt_log)$coefficients

slope <- coef_table_log_wt["log.weight.", "Estimate"]
slope
```

```
#> [1] -1.057506
```

The slope of log.weight. is -1.0575. Since this is a log-log regression, the slope is interpreted as a percentage relationship. A 1% increase in car weight is associated with about a 1.06% decrease in mpg.

(vi) From its standard error, what is the 95% confidence interval of the slope of log.weight. vs log.mpg.?

```
slope_se <- coef_table_log_wt["log.weight.", "Std. Error"]

lower_ci <- slope - 1.96 * slope_se
upper_ci <- slope + 1.96 * slope_se

c(lower_ci, upper_ci)
```

```
#> [1] -1.1157152 -0.9992977
```

The 95 percent confidence interval for the slope of log.weight. is approximately [-1.116, -0.999].

Q2 · Let's tackle multicollinearity next. Consider the regression model:

(a) Using regression and R^2, compute the VIF of log.weight. using the approach shown in class

```
# regress log.weight. on all other independent variables
weight_vif_regr <- lm(
  log.weight. ~ log.cylinders. + log.displacement. + log.horsepower. +
    log.acceleration. + model_year + factor(origin),
  data = cars_log
)

# get R-squared from this auxiliary regression
r2_weight <- summary(weight_vif_regr)$r.squared

# compute VIF
vif_weight <- 1 / (1 - r2_weight)

r2_weight
```

```
#> [1] 0.9431014
```

```
vif_weight
```

```
#> [1] 17.57512
```

(b) Let's try a procedure called Stepwise VIF Selection to remove highly collinear predictors. Start by Installing the 'car' package in RStudio – it has a function called vif()

(i) Use vif(regr_log) to compute VIF of the all the independent variables

```
library(car)

# full log model from Q1
regr_log <- lm(
  log.mpg. ~ log.cylinders. + log.displacement. + log.horsepower. +
    log.weight. + log.acceleration. + model_year + factor(origin),
  data = cars_log
)

# compute VIF values
vif(regr_log)
```

```
#>                       GVIF Df GVIF^(1/(2*Df))
#> log.cylinders.    10.456738  1        3.233688
#> log.displacement. 29.625732  1        5.442952
#> log.horsepower.   12.132057  1        3.483110
#> log.weight.       17.575117  1        4.192269
#> log.acceleration.  3.570357  1        1.889539
#> model_year         1.303738  1        1.141814
#> factor(origin)     2.656795  2        1.276702
```

(ii) Eliminate from your model the single independent variable with the largest VIF score that is also greater than 5

(iii) Repeat steps (i) and (ii) until no more independent variables have VIF scores above 5

```
# function to get comparable VIF values
get_vif_values <- function(model) {
  v <- vif(model)

  if (is.matrix(v)) {
    vif_compare <- ifelse(
     # Use ordinary GVIF for Df = 1 terms,
     # and adjusted GVIF for Df > 1 terms

      v[, "Df"] == 1,
      v[, "GVIF"],
      v[, "GVIF^(1/(2*Df))"]
```

```
    )
    names(vif_compare) <- rownames(v)
    return(vif_compare)
  } else {
    return(v)
  }
}

# start from the full model
current_model <- regr_log

# repeat until all VIF values are <= 5
repeat {
  vif_values <- get_vif_values(current_model)
  print(vif_values)

  max_vif <- max(vif_values)

  # stop if no VIF is above 5
  if (max_vif <= 5) {
    break
  }

  # find variable with largest VIF
  remove_var <- names(which.max(vif_values))
  cat("Removing:", remove_var, "with VIF =", max_vif, "\n")

  # update model by removing that variable
  new_formula <- update(formula(current_model), paste(". ~ . -", remove_var))
  current_model <- lm(new_formula, data = cars_log)
}
```

```
#>    log.cylinders. log.displacement.   log.horsepower.       log.weight.
#>         10.456738         29.625732         12.132057         17.575117
#> log.acceleration.        model_year    factor(origin)
#>          3.570357          1.303738          1.276702
#> Removing: log.displacement. with VIF = 29.62573
#>    log.cylinders.   log.horsepower.       log.weight. log.acceleration.
#>          5.433107         12.114475         11.239741          3.327967
#>        model_year    factor(origin)
#>          1.291741          1.173685
#> Removing: log.horsepower. with VIF = 12.11447
#>    log.cylinders.       log.weight. log.acceleration.        model_year
#>          5.427610          4.871730          1.401202          1.206351
#>    factor(origin)
#>          1.161682
#> Removing: log.cylinders. with VIF = 5.42761
```

```
#>       log.weight. log.acceleration.        model_year    factor(origin)
#>          1.933208          1.304761          1.175545          1.143564
```

```
# final model after stepwise VIF selection
final_model <- current_model
```

For predictors with Df = 1, the GVIF column is the same as the usual VIF, so I compare the GVIF column to the threshold of 5. For multi-degree-of-freedom terms such as factor(origin), I use GVIF^(1/(2*Df)) as the comparable value.

(iv) Report the final regression model and its summary statistics

```
# final model formula
formula(final_model)
```

```
#> log.mpg. ~ log.weight. + log.acceleration. + model_year + factor(origin)
```

```
# final model summary
summary(final_model)
```

```
#>
#> Call:
#> lm(formula = new_formula, data = cars_log)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.38259 -0.07054  0.00401  0.06696  0.39798
#>
#> Coefficients:
#>                     Estimate Std. Error t value Pr(>|t|)
#> (Intercept)         7.410974   0.316806  23.393  < 2e-16 ***
#> log.weight.        -0.875499   0.029086 -30.101  < 2e-16 ***
#> log.acceleration.   0.054377   0.037132   1.464  0.14389
#> model_year          0.032787   0.001731  18.937  < 2e-16 ***
#> factor(origin)2     0.056111   0.018241   3.076  0.00225 **
#> factor(origin)3     0.031937   0.018506   1.726  0.08519 .
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.1163 on 386 degrees of freedom
#> Multiple R-squared:  0.8845, Adjusted R-squared:  0.883
#> F-statistic: 591.1 on 5 and 386 DF,  p-value: < 2.2e-16
```

```
# final VIF values
get_vif_values(final_model)
```

```
#>       log.weight. log.acceleration.        model_year    factor(origin)
#>          1.933208          1.304761          1.175545          1.143564
```

(c) Using stepwise VIF selection, have we lost any variables that were previously significant? If so, how much did we hurt our explanation by dropping those variables? (hint: look at model fit)

```
# significant variables in the original full log model at 10%
full_sig <- summary(regr_log)$coefficients
full_sig[full_sig[, "Pr(>|t|)"] < 0.10, ]
```

```
#>                      Estimate  Std. Error   t value     Pr(>|t|)
#> (Intercept)        7.30193842 0.361776739 20.183549 3.164927e-62
#> log.horsepower.   -0.28475129 0.057944973 -4.914167 1.324103e-06
#> log.weight.       -0.59295537 0.085164828 -6.962444 1.461217e-11
#> log.acceleration. -0.16967287 0.059649081 -2.844518 4.686605e-03
#> model_year         0.03023926 0.001770634 17.078210 5.044732e-49
#> factor(origin)2    0.05071723 0.020919628  2.424385 1.579639e-02
#> factor(origin)3    0.04721481 0.020621538  2.289587 2.258853e-02
```

```
# significant variables in the final VIF-selected model at 10%
final_sig <- summary(final_model)$coefficients
final_sig[final_sig[, "Pr(>|t|)"] < 0.10, ]
```

```
#>                     Estimate  Std. Error    t value      Pr(>|t|)
#> (Intercept)      7.41097361 0.316806298  23.392760  5.343331e-76
#> log.weight.     -0.87549901 0.029085822 -30.100543 2.629486e-103
#> model_year       0.03278658 0.001731347  18.937038  4.951533e-57
#> factor(origin)2  0.05611103 0.018240925   3.076107  2.246666e-03
#> factor(origin)3  0.03193692 0.018505763   1.725782  8.518688e-02
```

```
# compare model fit before and after VIF selection
full_r2 <- summary(regr_log)$r.squared
final_r2 <- summary(final_model)$r.squared

full_adj_r2 <- summary(regr_log)$adj.r.squared
final_adj_r2 <- summary(final_model)$adj.r.squared

model_fit_compare <- data.frame(
  Model = c("Full log model", "Final VIF-selected model"),
```

```
  R_squared = c(full_r2, final_r2),
  Adjusted_R_squared = c(full_adj_r2, final_adj_r2)
)

model_fit_compare
```

```
#>                       Model R_squared Adjusted_R_squared
#> 1           Full log model 0.8919100          0.8896522
#> 2 Final VIF-selected model 0.8844856          0.8829893
```

> Using stepwise VIF selection, we did lose a variable that was previously significant.

Using stepwise VIF selection, we did lose a variable that was previously significant. The removed variables were log.displacement., log.horsepower., and log.cylinders.. Among these, log.horsepower. was previously significant in the full log model, with p-value = 1.32e-06. This means that stepwise VIF selection removed one predictor that was statistically significant in the full log model.

Also, log.acceleration. was significant in the original full log model, with p-value = 0.00469, but in the final VIF-selected model it was no longer significant, with p-value = 0.14389. It was not dropped from the model, but its effect became weaker after the highly collinear variables were removed.

To see how much this hurt the model, we compare model fit. The full log model had R-squared = 0.8919100 and adjusted R-squared = 0.8896522. The final VIF-selected model had R-squared = 0.8844856 and adjusted R-squared = 0.8829893. The adjusted R-squared decreased by:
0.8896522 - 0.8829893
= 0.0066629

So the explanatory power decreased by only about 0.0067, or 0.67 percentage points. This is a small drop.

> Therefore, even though we removed one previously significant variable, the final model still explains the data almost as well while having much lower multicollinearity.

This suggests that log.horsepower. contained useful information, but much of that information overlapped with the remaining predictors, especially log.weight.

(d) From only the formula for VIF, try deducing/deriving the following:

(i) If an independent variable has no correlation with other independent variables, what would its VIF score be?

```
R2_no_correlation <- 0

VIF_no_correlation <- 1 / (1 - R2_no_correlation)

VIF_no_correlation
```

```
#> [1] 1
```

If an independent variable has no correlation with the other independent variables, then the R-squared from regressing it on the other predictors is 0.

Using the VIF formula: VIF = 1 / (1 - R^2)
If $R^2 = 0$:
VIF = 1 / (1 - 0) = 1

> the VIF score would be 1.

This is the minimum possible VIF and means there is no multicollinearity.

(ii) Given a regression with only two independent variables (X1 and X2), how correlated would X1 and X2 have to be, to get VIF scores of 5 or higher? To get VIF scores of 10 or higher?

```
# For VIF = 5
r_for_vif_5 <- sqrt(1 - 1/5)

# For VIF = 10
r_for_vif_10 <- sqrt(1 - 1/10)

r_for_vif_5
```

```
#> [1] 0.8944272
```

```
r_for_vif_10
```

```
#> [1] 0.9486833
```

> To get a VIF of 5 or higher, the two independent variables need an absolute correlation of at least about 0.894. To get a VIF of 10 or higher, they need an absolute correlation of at least about 0.949.

This means the predictors must be extremely strongly correlated to create such high VIF values.

Q3 · Might the relationship of weight on mpg be different for cars from different origins? Let's try visualizing this. First, plot all the weights, using different colors and symbols for the three origins:

(a) Let's add three separate regression lines on the scatterplot, one for each of the origins. Here's one for the US to get you started:

```
origin_colors <- c("blue", "darkgreen", "red")

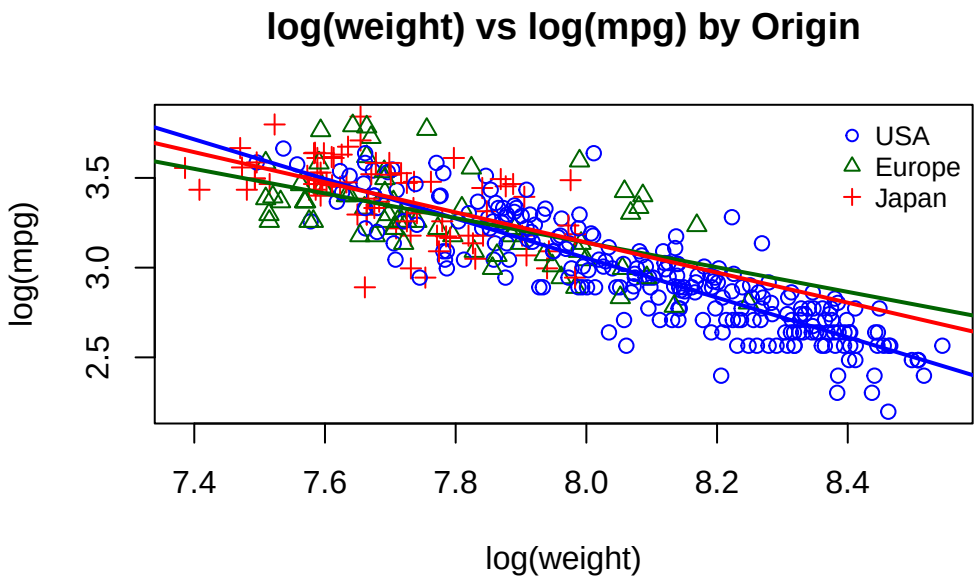
with(
  cars_log,
  plot(
    log.weight.,
    log.mpg.,
    pch = origin,
    col = origin_colors[origin],
    xlab = "log(weight)",
    ylab = "log(mpg)",
    main = "log(weight) vs log(mpg) by Origin"
  )
)

legend(
  "topright",
  legend = c("USA", "Europe", "Japan"),
  col = origin_colors,
  pch = 1:3,
  bty = "n",
  cex = 0.8
)

# separate datasets by origin
cars_us <- subset(cars_log, origin == 1)
cars_eu <- subset(cars_log, origin == 2)
cars_jp <- subset(cars_log, origin == 3)

# separate regressions for each origin
wt_regr_us <- lm(log.mpg. ~ log.weight., data = cars_us)
wt_regr_eu <- lm(log.mpg. ~ log.weight., data = cars_eu)
wt_regr_jp <- lm(log.mpg. ~ log.weight., data = cars_jp)

# add regression lines
abline(wt_regr_us, col = origin_colors[1], lwd = 2)
abline(wt_regr_eu, col = origin_colors[2], lwd = 2)
abline(wt_regr_jp, col = origin_colors[3], lwd = 2)
```



(b) Do cars from different origins appear to have different weight vs. mpg relationships?

Yes, cars from different origins appear to have somewhat different weight vs. mpg relationships. All three groups show a negative relationship, meaning heavier cars tend to have lower mpg. However, the regression lines do not perfectly overlap, so origin seems to affect the relationship between weight and mpg.

The main pattern is that cars from different origins may have different intercepts and possibly different slopes. This means that cars from one origin may have higher or lower expected mpg than cars from another origin at the same weight, and the strength of the weight effect may also differ slightly.